

改造 nachos 内核以支持多道程序

一、实验目的

本次实验的目的在于改造 nachos 内核以支持多道程序。实验内容分四部分：实现新的内存管理模块以完成基本分页管理；实现 Exec 系统调用；实现 Exit 和 Join 系统调用（选做）；编写简单的用户态测试程序以测试新改造的多道程序环境（详细内容请看 nachos-labs.pdf）。

二、实验内容

2.1 实现新的内存管理模块以支持基本分页管理

系统已经实现了分页管理的硬件支持——页表和地址变换机构，请参考 machine/translate.cc 和 machine/translate.h

目前的系统仅支持单道程序，因此尽管使用分页地址变换机构来进行地址重定位，但内存的分配和回收却采用单一连续分配。这点主要体现在 userprog/addrspace.cc 里 AddrSpace::AddrSpace 的实现。该实现中一旦创建新的用户程序，就从用户内存（实际上是一个字符数组 machine->mainMemory，见 machine/machine.h 和 machine/machine.h）地低端开始连续读入相应的内存映像。

我们要做的是改变这种局面，实现离散的内存分配和回收。

需要扩充的一点是：对用户内存分页框管理，引入某种机制对各个页框的使用情况进行登记。可以使用 bitmap 机制（userprog/bitmap.cc 和 userprog/bitmap.h）也可以如 nachos-labs.pdf 提示的那样使用 lab2 里所实现的 Table 机制。这部分代码请放在 userprog/memorymanager.cc 和 userprog/memorymanager.h 中

为了实现离散存储分配，注意在导入用户程序内存映像时应注意逐页导入。为了处理内存不足的错误，请将原来 AddrSpace::AddrSpace 里的代码改写后剥离出来成为 AddrSpace 的一个公共方法（请修改 userprog/addrspace.cc 和 userprog/addrspace.h 的相关部分）。

请修改 userprog/progtest.cc 的 StartProcess 函数以适应新的修改。并直接用 ttest/halt 进行测试（让 nachos 执行用户程序的步骤见 2.4）。

2.2 实现 Exec 系统调用

Exec 是一个系统调用，因此首先需要阅读的是 userprog/syscall.h，从中可以知 Exec 对应的系统调用号和该调用的原型。

系统调用的入口在 userprog/exception.cc 中，ExceptionHandler 函数将处理是异常处理和系统调用的总入口。目前仅包含对 Halt 系统调用的处理。我们所要添加的代码应从此函数引出（可使用 switch 进行分支）。

Exec 系统调用需要完成的工作是：根据给定的可执行文件名创建一个进程，并让进程执行该程序。

需要处理的第一件事是：获得可执文件名。请考虑如下几个问题：

当用户程序执行 Exec 系统调用时，文件名参数如何传递给内核？（请阅读 userprog/exception.cc 关于 ExceptionHandler 的注释）

作为参数的字符串首地址是否能直接用？请注意区分逻辑地址与物理地址、用户空间与内核空间的差别。值得强调的是：用户空间现在使用的是 machine->mainMemory，内核空间则直接使用系统内存。

各种出错情况都需要考虑，如：文件不存在、文件名超长等。

nachos 里的进程至少应包含一个线程。线程才是调度的单位。

理解 nachos 里进程和线程的区别：（见 threads/thread.h）

进程比线程多了一个地址空间，包含用户空间程序

进程比线程多了一组寄存器的值

进程比线程多了两个分别用于保存和恢复寄存器值的方法

userprog/progtest.cc 里的 StartProcess 函数提供了一些参考。但在参考时应该注意

Machine::Run 方法用于逐条执行进程的用户程序。

目前的实现中仅考虑单道程序，因而直接设置 currentThread->space。如果是多道程序还能这么做吗？

Exec 的返回值是进程地址空间的 id（0 表示错误），因而必须维护一个表以管理所有的进程。可以使用 lab2 里实现的 Table。另外，请注意一个细节，如何设置 Exec 的返回值？（请阅读 userprog/exception.cc 关于 ExceptionHandler 的注释）

值得特别注意的是，系统调用服务执行完毕后，硬件的指令计数器并没有递增。（见 machine/mipsim.cc 里的 Machine::OneInstruction 方法的实现）这将导致相应的系统调用被一再执行！可以通过添加如下函数并在系统调用执行完毕后调用该函数而实现相应计数器的递增：

```
void AdjustPCRegs()
{
    int pc;

    pc = machine->ReadRegister(PCReg);

    machine->WriteRegister(PrevPCReg, pc);

    pc = machine->ReadRegister(NextPCReg);

    machine->WriteRegister(PCReg, pc);

    pc += 4;

    machine->WriteRegister(NextPCReg, pc);
}
```

在正式实现 Exit 系统调用之前，也必须对 Exit 调用进行处理。可以简单地结束当前线程的执行（调用 `currentThread->Finish()`）。否则，无法正常对 Exec 的代码实现是否正确进行测试。

2.3 实现 Exit 和 Join 系统调用

Exit 调用应该能回收大部分相应进程的资源、记录退出状态并唤醒可能的等待者（调用了 Join 的父进程）

Exit 的代码应该实现为一个函数。这样，很多系统调用的出错处理都方便地调用之。

Join 调用后，相应进程将等待直到指定的子进程退出。

几个问题值得考虑：

Exit 的退出状态放在什么地方？

如果子进程先退出，父进程再调用 Join，则如何处理。

如果父进程没有调用 Join 则会不会有子进程的资源没有释放？

2.4 关于建立和执行新的测试程序

阅读 test/Makefile，了解 test 目录下的.c 文件是如何建立成能在 nachos (MIPS 的模拟平台) 中运行的可执行版本。

要建立新的测试程序可以遵循如下步骤（以 mytest.c 为例）：

编辑该测试程序，并保存在 test 目录中

修改 test/Makefile 里的关于目标 all 的规则，在 all 的前提条件里增加 mytest

在 test/Makefile 增加两条新的规则如下：

```
mytest.o: mytest.c
```

```
$(CC) $(CFLAGS) -c mytest.c
```

```
mytest: mytest.o start.o
```

```
$(LD) $(LDFLAGS) start.o mytest.o -o mytest.coff
```

```
../bin/coff2noff mytest.coff mytest
```

在 test 目录下键入 make 命令

编写 nachos 系统的用户程序时应注意

标准 c 的函数如：printf，scanf 等都不能用。

应使用 c 的语法。

如果觉得需要用户级的调试信息输出，可以考虑添加一个相应的系统调用。

执行测试程序请使用 userprog/下的 nachos 并带上-x 参数。如：
chenyd@cs8:~/nachos/nachos-3.4/code/userprog\$./nachos -x ../test/halt

需要编写的两个测试程序的要求请参考 nachos-labs.pdf 的 p29

三、实验思路

3.1 实现新的内存管理模块以支持基本分页管理

通过对 AddrSpace::AddrSpace 的改写予以实现，nachos 本身提供的虚存与实存的对应机制是——对应的
pageTable[i].physicalPage = i;我们所要做的就是将其改编成离散的内存分配和回收 pageTable[i].physicalPage =
memM->nextFree();并将 AddrSpace::AddrSpace 里的代码剥离出来成为 AddrSpace 的一个公共方法，改写代码如下：

将 AddrSpace::AddrSpace 剥离：

```
AddrSpace::AddrSpace(OpenFile *executable)
```

```
{
```

```
    execFile = executable;
```

```
}
```

新建方法 AddrSpace::NewPages(), 将改写后的 AddrSpace::AddrSpace 放在其中，代码如下：

```
if (memM->numFree() < numPages) {
```

```
    cout << "-- Not enough pages for process!!" << endl;
```

```
    cout << "-- Current freePageNum : " << memM->numFree()
```

```
    << ",the physical page needed: " << numPages
```

```
<< endl;
```

```
Exit(-127);
```

```
return;
```

```
}
```

```
pageTable = new TranslationEntry[numPages];
```

```
for (i = 0; i < numPages; i++){
```

```
    pageTable[i].virtualPage = i;    // for now, virtua page # != phys page #
```

```
    //
```

```
    pageTable[i].physicalPage = memM->nextFree();
```

```
    /*cout << " virtualPage: " << i
```

```
        << " physicalPage: " << pageTable[i].physicalPage
```

```
        << endl;*/
```

```
    pageTable[i].valid = TRUE;
```

```
pageTable[i].use = FALSE;
```

```
pageTable[i].dirty = FALSE;
```

```
pageTable[i].readOnly = FALSE; // if the code segment was entirely on
```

```
    // a separate page, we could set its
```

```
    // pages to be read-only
```

```
}
```

```
// then, copy in the code and data segments into memory (page by page)
```

```
int codeSize = noffH.code.size + noffH.initData.size;
```

```
char *buffer = new char[codeSize + 4]; // assume that int = 4 bytes
```

```
execFile->ReadAt(buffer, codeSize, noffH.code.inFileAddr);
```

```
//
```

```

int offset, page;

int vaddr = noffH.code.virtualAddr;

for (int j=0; j < codeSize; j++) {

    page = (int)(vaddr / PageSize);

    offset = vaddr % PageSize;

    /* cout << "vaddr:" << vaddr << " page:" << page

        << " offset:" << offset

        << endl;*/

    machine->mainMemory[pageTable[page].physicalPage*PageSize + offset] =

        buffer[j];

    vaddr++;

}

```

编写 memorymanager.cc 和 memorymanager.h 实现对用户内存分页框管理，对各个页框的使用情况进行登记。
Memorymanager.h 的类定义如下：

```

class MemoryManager {

```

public:

MemoryManager(); //构造函数,初始化 pageTable

~MemoryManager();

void mark(int pageNum); // allocate//标记哪个页已经被进程占有

void free(int pageNum); // free a frame//清空内存中的所有页

void busy(int pageNum); // in using ?

void print(); // current states of the allcated frames//打印被占的页号

boo test(int pageNum); // dirty or not//测试 which 页是否是脏页

int nextFree(); // find a free frame, and mark it //寻找一个空页

int numFree(); // number of tota free framse//总空闲页数

private:

```
    BitMap * bitTable;    //通过 pageTable 实现 bitmap 机制  
  
};
```

通过 memorymanager.cc 将 memorymanager.h 中的类接口和 bitmap 一一对应。

然后同实验一样改写 makefile 文件，在 usrprog 目录下 make 并键入 ./nachos -x ../test/halt 运行。在 system.cc 里面实例化了一个 MemoryManager 类的 memM 对象，提供对整个系统的内存分页分配和回收管理。

3.2 实现 Exec 系统调用

作为参数的字符串首地址能不能直接使用。通过读取寄存器来读入数据

对于有参数的系统调用，MIPS 编译决定了参数传递的以下规则：

参数 1: r4 寄存器

参数 2: r5 寄存器

参数 3: r6 寄存器

参数 4: r7 寄存器

如果系统调用有返回值，根据 MIPS 的标准 C 调用习惯，返回值存放在 r2 寄存器中。

于是改写 exception.cc 文件如下：

添加新的系统调用：

case SC_Exec :

```
    DEBUG('a', "Exec call\n");
```

```
    from = machine->ReadRegister(4);
```

```
for (i = 0; i < MAXLEN; i++) { //将文件名存入 fileName[] 中,每次读内存一个字节
```

```
    machine->ReadMem(from, 1, &tmp);
```

```
    fileName[i] = (char)tmp;
```

```
    if ((char)tmp == '\0')
```

```
        break;
```

```
    from++;
```

```
}
```

```
if (i == MAXLEN)
```

```
    cout << "the file name must be less than 128 chars!"
```

```
    << endl;
```

```
spaceId = new_Exec(fileName); //new_Exec()中传的是 fileName 所以要得到文件名,方法见上,spaceId 要保存返回
```

```
machine->WriteRegister(2, spaceId); //寄存器存返回值
```

```
pM->AdjustPCRegs();
```

```
break;
```

其中通过 pM 对象进行进程管理，其代码保存在 processmanager.cc 和 processmanager.h 中。

Processmanager.h 中定义的类如下：

```
typedef struct processCB {           // 进程控制块
```

```
    int spaceId;
```

```
    int numWait;
```

```
    int parent;
```

```
    int exitCode;
```

```
    Lock_sleep *lock;
```

```
    Condition_sleep *wait;
```

```
}PCB;
```

```
class ProcessManager {

public:

    ProcessManager();

    ~ProcessManager();

    // the process ID table

    int Alloc(PCB *object);

    PCB *Get(int spaceId);

    void Release(int id);

    void AdjustPCRegs(); // increase the pc after returns// 用于在调用增加 PC

private:

    Table *processTable;
```

```
};
```

3.3 实现 Exit 和 Join 系统调用

实现这些调用同 exec 是大同小异的，只需要在 exception.cc 中添加如下代码：

```
case SC_Exit :
```

```
    DEBUG('a', "Exit call\n");
```

```
    Status = machine->ReadRegister(4); //new_Exit()中传的是 Status 故只须从寄存器中取出状态标号即可.也是
```

```
    new_Exit(Status);
```

```
    pM->AdjustPCRegs();
```

```
    break;
```

```
case SC_Join :
```

```
    DEBUG('a', "Join call\n");
```

```
    id = machine->ReadRegister(4);
```

```
exitCode = new_Join(id);
```

```
machine->WriteRegister(2, exitCode);
```

```
pM->AdjustPCRegs();
```

`break;` 要建立新的测试程序可以遵循如下步骤（以 `mytest.c` 为例）：

3.4 编辑该测试程序，并保存在 test 目录中

编写两个测试函数如下：

tree.c :

```
int
```

```
main()
```

```
{
```

```
int i, j, k, tmp;
```

```
for (i = 0; i < M; i++){
```

```
    for (k = 0; k < N*i; k++){
```

```
        tmp = Exec("../test/run");
```

```
    Join(tmp);
```

```
    }
```

```
}
```

```
return 0;
```

```
}
```

tree-nojoin.c :

```
int
```

```
main()
```

```
{

    int i, j, k, tmp;

    for (i = 0; i < M; i++){

        for (k = 0; k < N*i; k++){

            tmp = Exec("../test/run");

        }

    }

    return 0;

}
```

四、实验结果

1、新内存分配机制，输入./nachos -x ../test/halt,得到结果：


```
[cs083825@cs8 userprog]$ ./nachos -x ../test/halt
Machine halting!

Ticks: total 22, idle 0, system 10, user 12
Disk I/O: reads 0, writes 0
Console I/O: reads 0, writes 0
Paging: faults 0
Network I/O: packets received 0, sent 0

Cleaning up...
```

2、然后测试 tree.c 的，输入./nachos -x ../test/tree 测试结果：

取部分截图如下：

从图中，我们可以很明显的看到，调用 join 的父进程，在所有的子进程结束之后才结束的。

```
[cs083825@cs8 userprog]$ ./nachos -x ../test/tree
Exec :
New Process, ID : 1

Exec :
New Process, ID : 2

Join!

    test running

Exit!
Process 1 is end, returns : 0

    test running

Exit!
Process 2 is end, returns : 0

Exit!
Process 0 is end, returns : 0
No threads ready or runnable, and no pending interrupts.
Assuming the program completed.
Machine halting!

Ticks: total 352, idle 0, system 260, user 92
Disk I/O: reads 0, writes 0
Console I/O: reads 0, writes 0
Paging: faults 0
Network I/O: packets received 0, sent 0

Cleaning up...
```

3、接着对 tree-nojoin.c 进行测试，输入./nachos -x ../test/tree-nojoin, 得到的测试结果：

部分截图如下：

```
[cs083825@cs8 userprog]$ ./nachos -x ../test/tree-nojoin
Exec :
New Process, ID : 1

Exec :
New Process, ID : 2

Exit!
Process 0 is end, returns : 0

    test running

Exit!
Process 1 is end, returns : 0

    test running

Exit!
Process 2 is end, returns : 0
No threads ready or runnable, and no pending interrupts.
Assuming the program completed.
Machine halting!

Ticks: total 275, idle 0, system 190, user 85
Disk I/O: reads 0, writes 0
Console I/O: reads 0, writes 0
Paging: faults 0
Network I/O: packets received 0, sent 0

Cleaning up...
```

从图中，我们可以很明显的看出，没有调用 join 的父进程，在开启另外的两个进程之后，就结束了。

五、实现小结

本次实验的难度是相当大的，其中特别是我们新的内存与虚存的映射分配的代码编写，以及进程管理的编写我们要改在 nachos 的内存分配机制以支持多道程序，并实现 exec 和 join 以及 exit 的系统调用。

六、文件清单

~/nachos/nachos-3.4/code/Makefile.common

~/nachos/nachos-3.4/code/threads/main.cc

~/nachos/nachos-3.4/code/threads/system.h

~/nachos/nachos-3.4/code/threads/system.cc

~/nuchos/nuchos-3.4/code/userprog/progtest.cc
~/nuchos/nuchos-3.4/code/userprog/addrspace.h
~/nuchos/nuchos-3.4/code/userprog/addrspace.cc
~/nuchos/nuchos-3.4/code/userprog/memorymanager.h
~/nuchos/nuchos-3.4/code/userprog/memroymanager.cc
~/nuchos/nuchos-3.4/code/userprog/syscall.h
~/nuchos/nuchos-3.4/code/userprog/exception.cc
~/nuchos/nuchos-3.4/code/userprog/processmanager.h
~/nuchos/nuchos-3.4/code/userprog/processmanager.cc
~/nuchos/nuchos-3.4/code/test/tree-nojoin.c
~/nuchos/nuchos-3.4/code/test/tree.c
~/nuchos/nuchos04.doc

其中, nachos04.doc 为实验报告。

<script type="text/javascript" src="http://pagead2.googlesyndication.com/pagead/show_ads.js"></script>